Design Document for CyVal

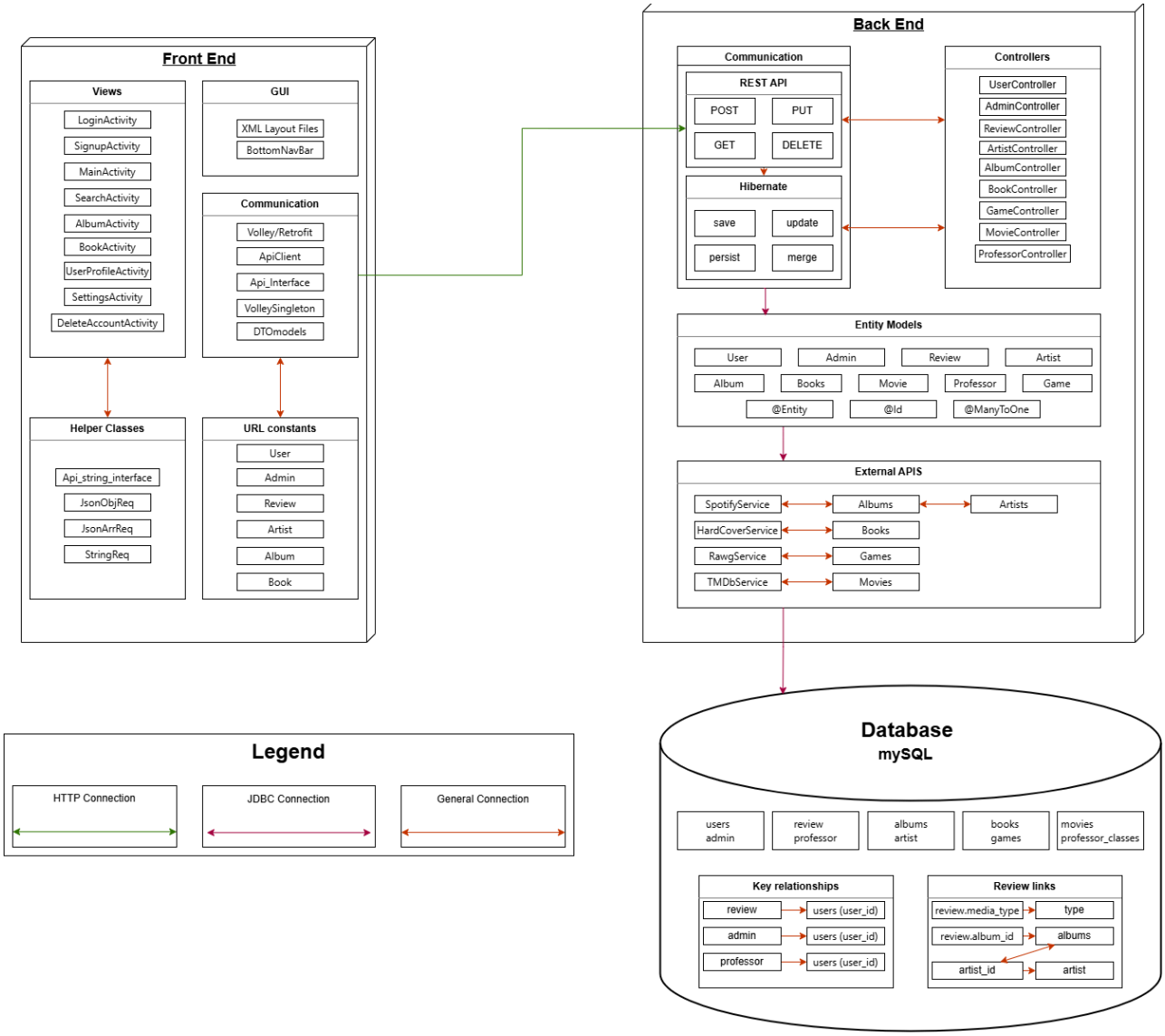
Group 2-SC-4

Kamil Halupka: 25% contribution

Alexander Tran: 25% contribution

Edwin Cepeda: 25% contribution

Cristian Alvarez: 25% contribution


Front End
Views
LoginActivity
SignupActivity
MainActivity
SearchActivity
AlbumActivity
BookActivity
UserProfileActivity
SettingsActivity
DeleteAccountActivity
GUI
XML Layout Files
BottomNavBar
Communication
Volley/Retrofit
ApiClient
Api_Interface
VolleySingleton
DTOmodels
Helper Classes
Api_string_interface
JsonObjReq
JsonArrReq
StringReq
URL constants
User
Admin
Review
Artist
Album
Book
Back End
Communication
REST API
POST
PUT
GET
DELETE
Hibernate
save
update
persist
merge
Controllers
UserController
AdminController
ReviewController
ArtistController
AlbumController
BookController
GameController
MovieController
ProfessorController
Entity Models
User
Admin
Review
Artist
Album
Books
Movie
Professor
Game
@Entity
@Id
@ManyToOne
External APIS
SpotifyService
Albums
Artists
HardCoverService
Books
RawgService
Games
TMDbService
Movies
Database
mySQL
users
admin
review
professor
albums
artist
books
games
movies
professor_classes
Key relationships
review
users (user_id)
admin
users (user_id)
professor
users (user_id)
Review links
review.media_type
type
review.album_id
albums
artist_id
artist
Legend
HTTP Connection
JDBC Connection
General Connection

Frontend (Currently Implemented)

Signup

- This page contains all the necessary functionality to register a new user. User type will be a standard user (student) by default until promoted by an administrator.
    - EditText: usernameEdit
    - EditText: passwordEdit
    - EditText: emailEdit
    - Button: signupButton

Login

- This page allows users to select their user type and then log in with their username and password.
    - Buttons: studentButton, professorButton, adminButton
    - EditText: usernameTextEdit
    - EditText: passwordTextEdit
    - Button: loginButton
- What usertype a user selects will affect the login logic when loginButton is pressed. If you're trying to login as an administrator and there's no admin ID associated with your user, it will reject you, and vice versa for trying to log in as a standard user when there *is* an admin ID attached to that user. Obviously, the username and password always need to be correct for the login to be successful.

Search

- This page allows any user to search for a piece of media by its title and its media type (movies, etc).
    - Spinner: mediaSelector (for selecting media type)
    - EditText: editMedia (for typing in title)
    - Button: searchButton
- searchButton performs a 2-stage search: First, it searches the respective database for the desired piece of media by the title in editMedia (for faster lookups if the media has already been searched for before), and if it can't find it, it searches an external database from an archival organization (such as OpenLibrary, for books) to pull information about that piece of media and add it to our database, before finally showing the results.

Backend

Communication

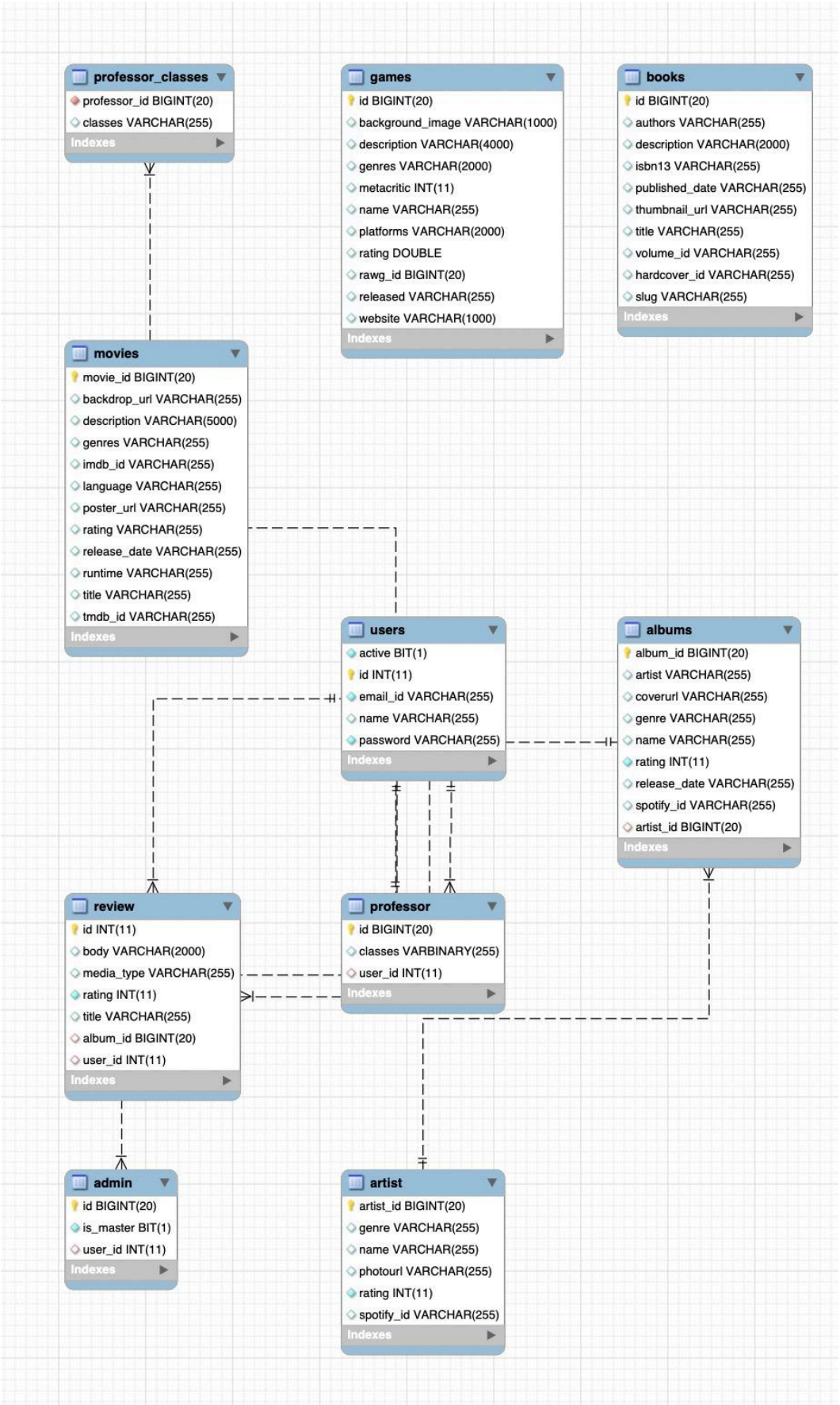
To facilitate the communication between the frontend and backend, and backend and database, we use a Tomcat server through SpringBoot. Each different module has its own controller with its own set of endpoints that are used to transfer data to the backend, which is then sent and saved in the database. Each REST operation is generally used as followed:

- GET: Used to retrieve data from the MySQL database. The backend performs database operations, and formats the return value as a ResponseEntity object, containing the appropriate object and an HTTP status code.
- POST: Used to save data to the MySQL database. Data is sent as a JSON object from the frontend, parsed and turned into a Java object in the backend, and is saved to the database. The return value consists of ResponseEntity containing a message String and an HTTP status code.
- PUT: Used to update existing data in the MySQL database. Data is sent as a JSON object, parsed and formatted to the proper Java object, and the changed field is updated and saved to an existing database entry. The return value consists of ResponseEntity containing a message String and an HTTP status code.
- DELETE: Used to delete existing data from the database. An identifier parameter, such as an ID, is sent to the backend from the frontend, with which the backend retrieves and deletes an existing entry from the database. The return value consists of ResponseEntity containing a message String and an HTTP status code.

Controllers

Our backend consists of the following controllers that are used to handle different endpoints:

- Users: Used for creation, retrieval, updating, and deletion of users. Handles signup and login for user accounts.
- Review: Used for creation, viewing, editing, and deletion of user reviews. Contains different parts, such as a body, media type, and score.
- Artists/Albums/Games/Movies/Books: Used for creation, viewing, editing, and deletion of supported media types for reviews. Used to allow users to search for and write reviews of different media types.
- Admin/Professor: Used for creation, retrieval, editing, and deletion of different user types. Used to check whether a user is of a specific type to support certain type-locked operations or features.


professor_classes
professor_id BIGINT(20)
classes VARCHAR(255)
Indexes
games
id BIGINT(20)
background_image VARCHAR(1000)
description VARCHAR(4000)
genres VARCHAR(2000)
metacritic INT(11)
name VARCHAR(255)
platforms VARCHAR(2000)
rating DOUBLE
rawg_id BIGINT(20)
released VARCHAR(255)
website VARCHAR(1000)
Indexes
books
id BIGINT(20)
authors VARCHAR(255)
description VARCHAR(2000)
isbn13 VARCHAR(255)
published_date VARCHAR(255)
thumbnail_url VARCHAR(255)
title VARCHAR(255)
volume_id VARCHAR(255)
hardcover_id VARCHAR(255)
slug VARCHAR(255)
Indexes
movies
movie_id BIGINT(20)
backdrop_url VARCHAR(255)
description VARCHAR(5000)
genres VARCHAR(255)
imdb_id VARCHAR(255)
language VARCHAR(255)
poster_url VARCHAR(255)
rating VARCHAR(255)
release_date VARCHAR(255)
runtime VARCHAR(255)
title VARCHAR(255)
tmdb_id VARCHAR(255)
Indexes
users
active BIT(1)
id INT(11)
email_id VARCHAR(255)
name VARCHAR(255)
password VARCHAR(255)
Indexes
albums
album_id BIGINT(20)
artist VARCHAR(255)
coverurl VARCHAR(255)
genre VARCHAR(255)
name VARCHAR(255)
rating INT(11)
release_date VARCHAR(255)
spotify_id VARCHAR(255)
artist_id BIGINT(20)
Indexes
review
id INT(11)
body VARCHAR(2000)
media_type VARCHAR(255)
rating INT(11)
title VARCHAR(255)
album_id BIGINT(20)
user_id INT(11)
Indexes
professor
id BIGINT(20)
classes VARBINARY(255)
user_id INT(11)
Indexes
admin
id BIGINT(20)
is_master BIT(1)
user_id INT(11)
Indexes
artist
artist_id BIGINT(20)
genre VARCHAR(255)
name VARCHAR(255)
photourl VARCHAR(255)
rating INT(11)
spotify_id VARCHAR(255)
Indexes